Caching System Design

Interview Preparation Guide · SDE → Principal

System Design · Distributed Systems · Performance Engineering

SDE (L3–L4)	Senior SDE (L5)	Staff Engineer (L6)	Principal Engineer (L7+)

Level Expectations at a Glance

Topic	SDE	Senior	Staff	Principal
Cache Basics	Define & use	Choose patterns	Design strategy	Org-wide vision
Eviction Policies	LRU / LFU	Compare tradeoffs	Custom policies	Research-level
Consistency	Eventual basics	Coherence models	Distributed CAP	Novel guarantees
Scale & Capacity	Basic sizing	Working sets	Fleet planning	Cost modeling
Failure Handling	Retry logic	Fallback patterns	Chaos resilience	Platform SLOs
Observability	Hit/miss rate	P99 latency	Anomaly detection	Infra-wide metrics
Security	Awareness	Data sanitization	Threat modeling	Policy framework

What interviewers expect: You can explain what a cache is, correctly use common caching patterns in code, and reason about basic tradeoffs.

1. Core Concepts

- What is a cache and why do we use it? (latency reduction, DB offloading)
- Cache hit vs cache miss — what happens in each case
- Hot data vs cold data — intuition for what to cache
- TTL (Time-To-Live) — setting expiration and why it matters

💡 *Be able to sketch a read path: Request → Cache → (miss) → DB → Cache write*

2. Eviction Policies

- LRU (Least Recently Used) — how it works, when to use
- LFU (Least Frequently Used) — difference from LRU
- FIFO — simple but often suboptimal
- Random eviction — sometimes surprisingly competitive

💡 *Know how to implement LRU using a HashMap + Doubly Linked List*

3. Caching Patterns

- Cache-Aside (Lazy Loading) — app checks cache, falls back to DB, populates cache
- Read-Through — cache sits in front, automatically loads from DB on miss
- Write-Through — write to cache and DB simultaneously
- Write-Back (Write-Behind) — write to cache, async flush to DB

💡 *Cache-aside is the most commonly used pattern in practice. Know its stale data risk.*

4. Where Caches Live

- In-process / in-memory (e.g., a HashMap in your service)
- Sidecar / local Redis / Memcached
- Distributed cache (shared across service instances)
- CDN / edge caching for static assets

5. Basic Consistency

- Eventual consistency — cache may be stale for TTL duration
- Cache invalidation on write — how and when to evict or update
- Stale reads — product-level tolerance for stale data

6. Interview Questions at This Level

- Design a key-value store with a cache layer
- How would you add caching to reduce DB load for a read-heavy API?
- What is cache invalidation and why is it hard?
- Implement LRU cache in code

What interviewers expect: You can architect caching for a production system, reason about tradeoffs, and discuss failure modes independently.

1. Distributed Caching

- Redis Cluster vs Memcached — consistency, data structures, persistence
- Consistent hashing — how cache keys map to nodes, virtual nodes
- Replication in Redis — primary/replica, sentinel, cluster modes
- Partitioning strategies — hash-based, range-based, directory-based

💡 *Know: Redis supports rich data types (sorted sets, streams); Memcached is simpler/faster for plain KV*

2. Cache Stampede / Thundering Herd

- Problem: many concurrent misses hitting the DB simultaneously
- Solutions: mutex/lock-based refresh, probabilistic early expiration (XFetch)
- Request coalescing — deduplicate concurrent misses into one DB fetch
- Background refresh — proactive TTL renewal before expiry

3. Hot Key Problem

- A single key receiving extreme request volume (e.g., celebrity post)
- Solution 1: Local in-process cache layer (L1) in front of Redis (L2)
- Solution 2: Key sharding — fan out reads to N copies (key:0 to key:N-1)
- Solution 3: Read replicas for hot Redis slots

4. Cache Sizing and Capacity Planning

- Working set analysis — what % of data is actually "hot"
- Cache hit rate as a function of cache size (Zipf distribution)
- Memory estimation: key count × avg entry size + overhead
- Eviction rate monitoring to detect undersized caches

💡 *Rule of thumb: 20% of data often accounts for 80% of reads. Size accordingly.*

5. Write Strategies Deep Dive

- Write-through tradeoff: safe but double-write latency for every write
- Write-behind: high throughput but risk of data loss on crash
- Choosing write strategy based on read:write ratio and durability requirements

6. Negative Caching

- Cache "miss" results (e.g., user not found = null) to prevent DB hammering
- Use a short TTL for negative entries to avoid stale negatives
- Differentiate between "not found" and "not cached yet"

7. CDN Caching

- Cache-Control headers: max-age, s-maxage, no-cache, no-store, must-revalidate
- ETags and conditional requests (If-None-Match) for efficient validation
- CDN purging strategies — full purge vs tag-based vs URL-based
- Origin shield — collapsing CDN misses before hitting origin

8. Interview Questions at This Level

- Design a distributed rate limiter using Redis
- How do you cache a Twitter/Instagram timeline? Fan-out on write vs read?

- Design a cache for a product catalog with 10M items
- How do you handle cache invalidation across microservices?

What interviewers expect: You drive caching strategy across services, make principled tradeoffs, reason about correctness under failure, and lead technical direction.

1. Multi-Tier Caching Architecture

- L1 (in-process): Guava Cache, Caffeine — sub-millisecond, no network
- L2 (sidecar/local Redis): single-node, shared within host
- L3 (distributed Redis/Memcached): shared across fleet
- L4 (CDN): geographic distribution, edge caching
- Designing promotion/demotion policies between tiers

💡 *Staff-level: articulate when each tier is appropriate and design the coherence protocol between them*

2. Consistency Models and CAP Tradeoffs

- Strong consistency — synchronous invalidation across all nodes (high latency cost)
- Eventual consistency — async propagation with bounded staleness window
- Read-your-writes consistency — guarantees within a session or user context
- Monotonic read consistency — user never sees older data after seeing newer
- How to define SLAs: "data may be stale up to X seconds"

3. Cache Coherence in Distributed Systems

- Broadcast invalidation — publish invalidation events to all nodes via pub/sub
- Tag-based invalidation — group related entries under a tag, invalidate by tag
- Version vectors — track cache version per entity, reject stale writes
- Two-phase invalidation — mark-then-delete to prevent race conditions

4. Cache Warming Strategies

- Cold start problem — empty cache causes DB overload on deployment
- Pre-warming: populate cache from DB dump or snapshot before traffic
- Gradual traffic shifting — bring up new instances with warming enabled
- Cache seeding from access logs — replay historical requests
- Warm transfer between cache instances during rolling deploys

5. Failure Modes and Resilience

- Cache failure → fallback to DB with circuit breaker pattern
- Cache poisoning — serving incorrect data; validate before caching
- Network partition between app and cache — stale-while-revalidate pattern
- Redis failover — Sentinel vs Cluster automatic failover timings
- Cascading failure — cache miss storm → DB overload → more misses → full outage

💡 *Staff-level: walk through failure scenarios end-to-end and describe your mitigation for each*

6. Observability and Monitoring

- Key metrics: hit rate, miss rate, eviction rate, latency (p50/p95/p99), memory utilization
- Per-key observability for hot key detection
- Alerting on hit rate drop > threshold (indicates cache eviction or invalidation storm)
- Cache efficiency dashboards — cost per request served, bytes per hit
- Tracing cache interactions with distributed tracing (Jaeger, Zipkin, X-Ray)

7. Caching for Specific System Design Scenarios

- Social media feed: precomputed timelines vs on-demand fan-in; celebrity account handling

- Search autocomplete: Redis sorted sets with score = frequency; trie in-memory
- Session management: Redis with TTL per session, sliding expiry
- E-commerce inventory: short TTL on stock counts, write-through for accuracy
- API gateway cache: Varnish, Nginx proxy cache, response caching with Vary headers

8. Caching and Database Interaction

- Cache-DB consistency: dual-write atomicity problem (write cache + DB, one fails)
- Using DB triggers or CDC (Change Data Capture) to drive cache invalidation
- Read replicas as a "poor man's cache" — when is this better?
- CQRS + cache: separate read models pre-materialized for query patterns

9. Interview Questions at This Level

- Design a caching platform/service that 50 microservices can use
- How do you migrate a live system from no-cache to a caching layer with zero downtime?
- You have a cache hit rate of 60% in production. How do you diagnose and fix it?
- Design the caching strategy for a payments system where stale data = real money risk

What interviewers expect: You shape caching strategy at org or industry scale, make research-informed decisions, evaluate build vs buy, and define standards adopted company-wide.

1. Caching as a Platform

- Building internal caching as a service — shared infrastructure vs per-team
- Multi-tenancy in caching: namespace isolation, quota enforcement, noisy neighbor
- API design for a caching platform: eviction policies as config, TTL defaults, invalidation contracts
- Versioned cache schemas — deploying cache schema changes without breaking consumers
- SLA definition: latency p99, availability, durability, consistency guarantees

2. Advanced Eviction and Admission Policies

- TinyLFU (Tiny Least Frequently Used) — probabilistic frequency counting, used in Caffeine
- SLRU (Segmented LRU) — probationary + protected segments to prevent scan pollution
- W-TinyLFU — windowed TinyLFU, best-in-class for general workloads (Caffeine default)
- Admission filters — bloom filters to prevent one-hit wonders polluting the cache
- Scan-resistant caching — file system-style scans should not evict hot data

💡 *At principal level, you should know the academic literature (e.g., Cliffhanger, ARC, LIRS papers)*

3. Geo-Distributed Caching

- Multi-region cache consistency — cross-region replication lag and conflict resolution
- Cache routing: route reads to nearest region, writes to primary with async replication
- Edge caching at PoPs (Points of Presence) — consistency vs performance tradeoff by geography
- Cache federation — independent regional caches vs globally consistent cache
- Data residency constraints — regulatory requirements (GDPR, CCPA) affecting cache placement

4. Cost Modeling and Efficiency

- Total cost of caching: memory cost, network cost, engineering overhead, operational burden
- ROI analysis: cache cost vs compute cost to regenerate data vs DB query cost
- Cache efficiency ratio: (data served from cache × compute saved) / cache infrastructure cost
- Rightsizing caches across a fleet using working set modeling and actual hit curves
- Autoscaling caches: predictive scaling vs reactive scaling based on miss rate

5. Consistency at Extreme Scale

- Lease-based consistency — clients hold leases; cache invalidations wait for leases to expire
- Memcache at Facebook: leases to solve stale sets, thundering herd, and stale read window
- Bounded staleness — define maximum tolerable staleness per entity type in SLA
- Version-stamped caches — every cache write includes a monotonically increasing version
- Conflict-free Replicated Data Types (CRDTs) for eventually consistent cached aggregates

6. Security and Compliance

- Cache poisoning attack vectors — input validation, Content Security Policy, signed cache tokens
- Sensitive data in cache — PII handling, encryption at rest/in-transit, field-level redaction
- Audit logging — who read what from cache, when (for compliance and forensics)
- Data classification: which data classes may be cached, for how long, in which tiers
- Cache access control — ACL per cache namespace, service identity-based authorization

7. Research and Emerging Patterns

- Predictive prefetching — ML models predicting what to cache before it's requested

- Semantic caching — cache based on query semantics, not just key equality (LLM query caching)
- Persistent memory (Intel Optane) caches — byte-addressable, DRAM-speed, disk-capacity
- Disaggregated caching — network-attached memory pools (e.g., CXL-based architectures)
- Cache-oblivious algorithms — algorithm design that performs optimally without cache knowledge

8. Build vs Buy Decisions

- Redis (open source) vs Redis Enterprise vs Elasticache vs Memorystore — TCO analysis
- When to build a custom caching layer vs adopt an existing solution
- Evaluating Varnish, Nginx, Squid, Envoy proxy caches for different use cases
- In-house cache engines (e.g., Pelikan at Twitter, Mcrouter at Facebook) — when justified

9. Interview Questions at This Level

- How would you design the caching strategy for a company moving from monolith to microservices?
- A major streaming platform has 500M users. Design their caching infrastructure end-to-end.
- The cache hit rate for one geography is 30% lower than others. How do you investigate?
- How do you define and enforce caching standards across 200 engineering teams?
- Describe a caching innovation you would contribute to the industry — what problem does it solve?

Quick Reference: Key Tradeoffs

Topic	SDE	Senior	Staff	Principal
Cache-Aside vs Read-Through	Know both	Choose per use case	Define team standard	Platform default policy
Write-Through vs Write-Behind	Know tradeoffs	Apply per durability need	Risk modeling	Compliance + perf balance
LRU vs LFU	Implement LRU	Choose by workload	Custom policies	W-TinyLFU / research
Redis vs Memcached	Basic difference	Choose & configure	Fleet standardization	Build vs buy + TCO
Strong vs Eventual Consistency	Define both	Apply CAP reasoning	Per-entity SLA	Platform consistency model
TTL vs Event-Driven Invalidation	Use TTL	Event-driven for freshness	Hybrid strategy	Contract-based invalidation
Single vs Multi-Tier	Single tier	Two tiers	Design multi-tier	Fleet-wide tier strategy

Top 5 System Design Scenarios

Topic	SDE	Senior	Staff	Principal
Twitter Feed	Simple DB + cache	Fan-out strategies	Celeb handling + tiers	Global consistency + cost
Rate Limiter	Redis INCR + TTL	Sliding window	Distributed + quota service	Multi-region + fairness
Autocomplete	Sorted set	Trie + Redis	Personalization + scale	ML-driven predictions
Product Catalog	TTL cache	Invalidation on update	CDN + app + DB tiers	Real-time inventory accuracy
Session Store	Redis KV + TTL	Sticky vs distributed	Multi-region sessions	Privacy + residency + scale

Good luck with your interview preparation!